COBOL Basics 1

COBOL coding rules

Almost all COBOL compilers treat a line of COBOL code as if **it contained two distinct areas.** These are known as;

Area A and Area B

When a COBOL compiler recognizes these two areas, all **division, section, paragraph names,** FD entries and 01 level **numbers must start in Area A.** All other sentences must start in **Area B.**

Area A is four characters wide and is followed by Area B.

In Microfocus COBOL the compiler directive

$ SET SOURCEFORMAT"FREE" frees us from all formatting restrictions.

COBOL coding rules

Almost all COBOL compilers treat a line of COBOL code as if it contained two distinct areas. These are known as;

Area A and Area B

When a COBOL compiler recognizes these two areas, all division, section, paragraph names, FD entries and 01 level numbers must start in Area A. All other sentences must start in Area B.

Area A is four characters wide and is followed by Area B.

In Microfocus COBOL the compiler directive

`$ SET SOURCEFORMAT"FREE"` frees us from all formatting restrictions.

```
     $ SET SOURCEFORMAT"FREE"
IDENTIFICATION DIVISION.
PROGRAM-ID.  ProgramFragment.
* This is a comment. It starts
* with an asterisk in column 1
```

Name Construction.

All user defined names, such as data names, paragraph **names, section names** and mnemonic names, must **adhere to the following** rules;

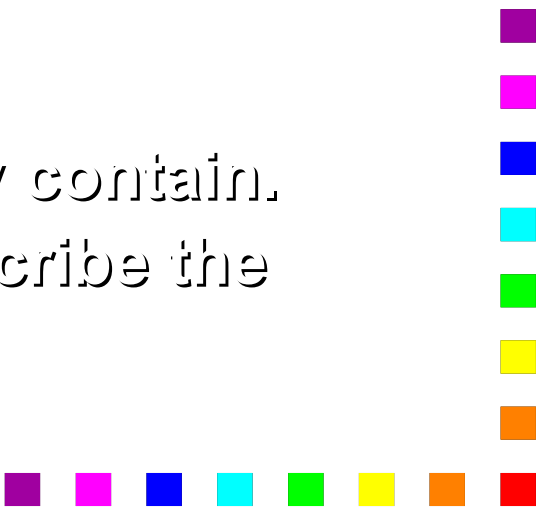
They must contain at least o
30 characters.

They must contain at least o
must not begin or end wit

Th

All data-names should describe the data they contain.

All paragraph and section names should describe the function of the paragraph or section.

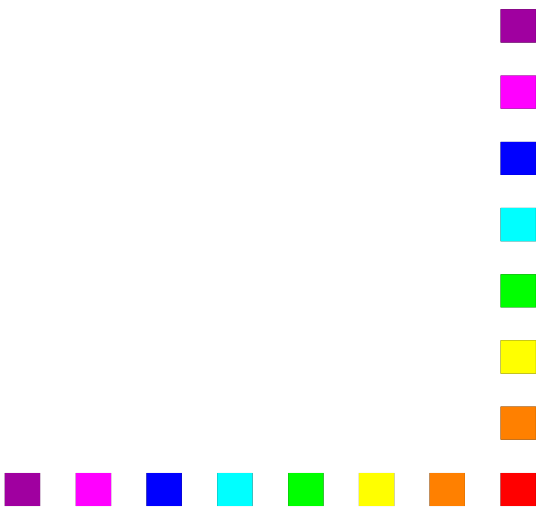


Describing DATA.

There are basically three kinds of data in COBOL programs;

1. Variables.
2. Literals.
3. Figurative Constants.

Unlike other programming languages, COBOL does not support user defined constants.



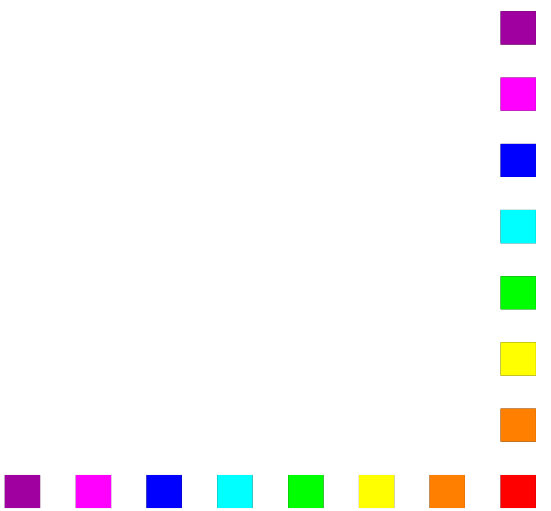
Data-Names / Variables

A **variable** is a named location in memory into which a program can put data and from which it can retrieve data.

A **data-name** or **identifier** is the name used to identify the area of memory reserved for the variable.

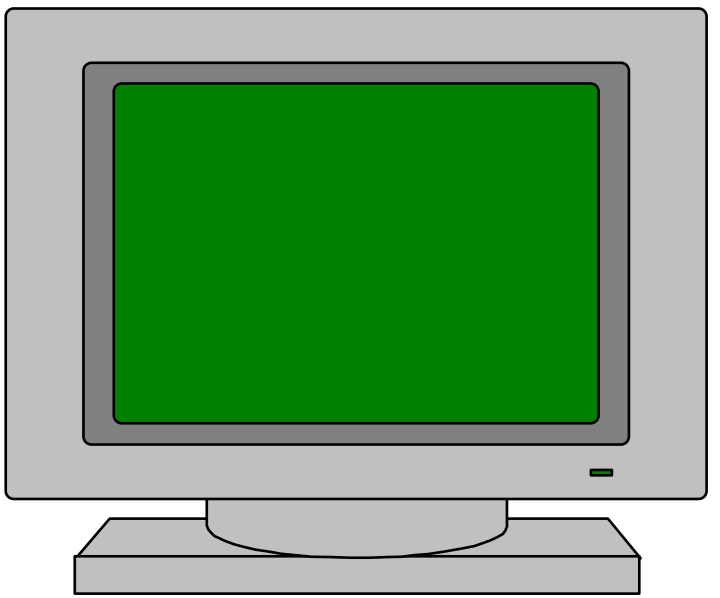
Variables must be described in terms of their type and size.

Every variable used in a COBOL program must have a description in the DATA DIVISION.



Using Variables

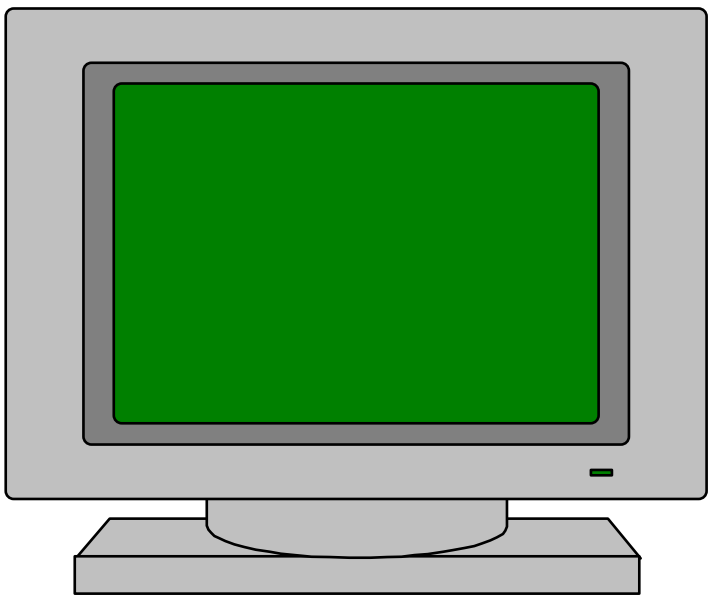
```
01 StudentName  PIC X(6) VALUE SPACES.

MOVE "JOHN" TO StudentName.
DISPLAY "My name is ", StudentName.
```



Using Variables

```
01 StudentName  PIC X(6) VALUE SPACES.

MOVE "JOHN" TO StudentName.
DISPLAY "My name is ", StudentName.
```

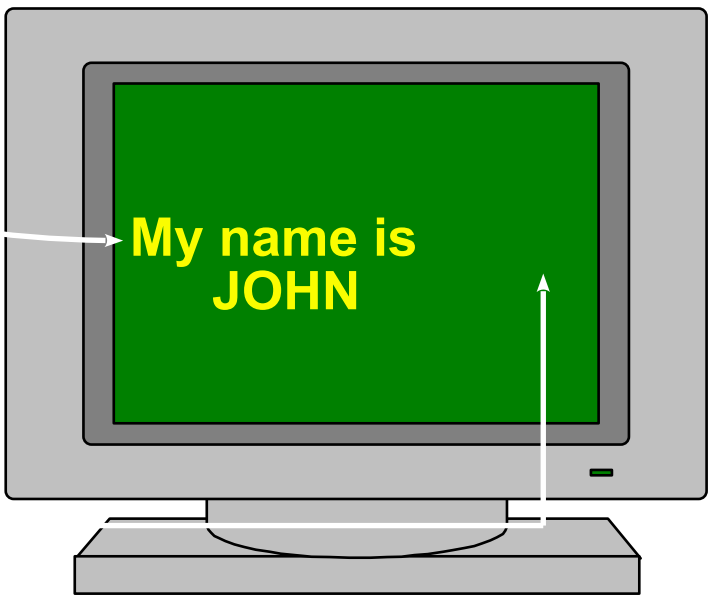
J	O	H	N		



Using Variables

```
01 StudentName  PIC X(6) VALUE SPACES.

MOVE "JOHN" TO StudentName.
DISPLAY "My name is ", StudentName.
```

J	O	H	N		



COBOL Data Types

COBOL is not a "typed" language and the distinction between some of the data types available in the language is a little blurred.

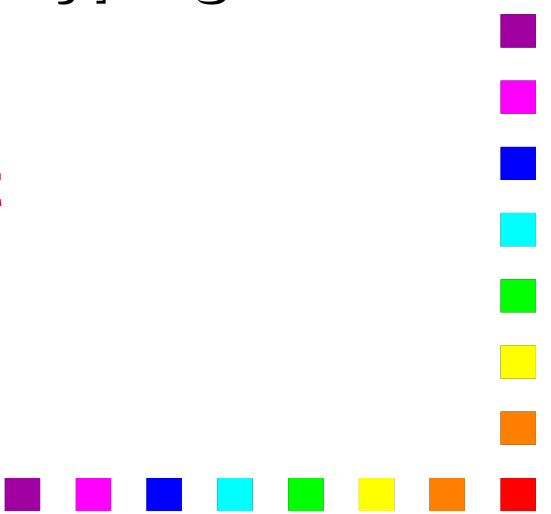
For the time being we will focus on just two data types,

- **numeric**
- **text or string**

Data type is important because it determines the operations which are valid on the type.

COBOL is not as rigorous in the application of typing rules as other languages.

not



Quick Review of "Data Typing"

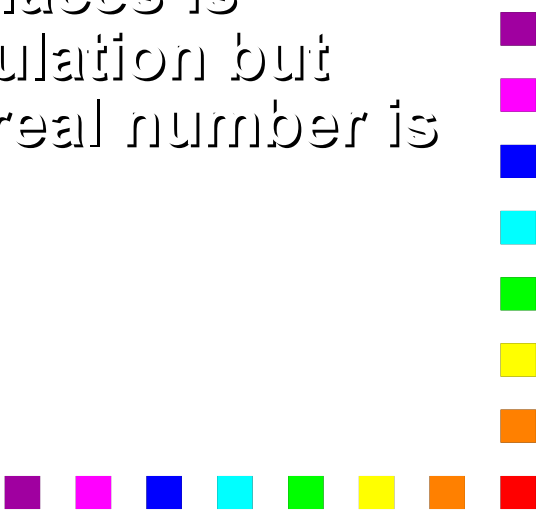
In "typed" languages simply specifying the type of a data item provides quite a lot of information about it.

The type usually determines the range of values the data item can store.

For instance a CARDINAL it
0..65,535 and an INTEGE

From the type of the item the compiler can establish how much memory to set aside for storing its values.

If the type is "REAL" the number of decimal places is allowed to vary dynamically with each calculation but the amount of the memory used to store a real number is fixed.



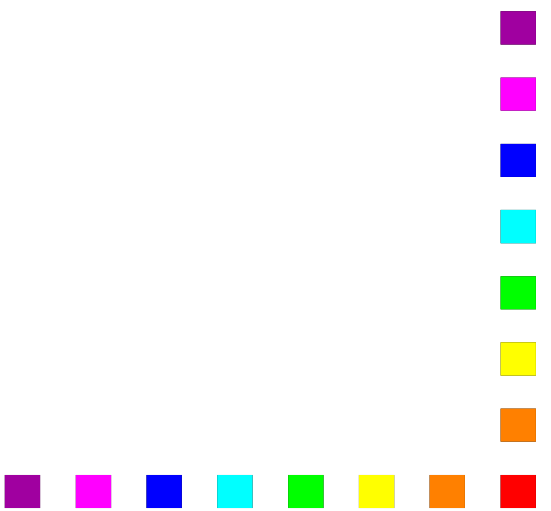
COBOL data description.

Because **COBOL is not typed** it employs a different mechanism for describing the characteristics of the data items in the program.

COBOL uses what could be described as a "**declaration by example**" strategy.

In effect, the programmer provides the system with an example, or template, or **PIC**TURE of what the data item looks like.

From the "picture" the system derives the information necessary to allocate it.



COBOL ‘PICTURE’ Clause symbols

To create the required ‘picture’ the programmer uses a set of symbols.

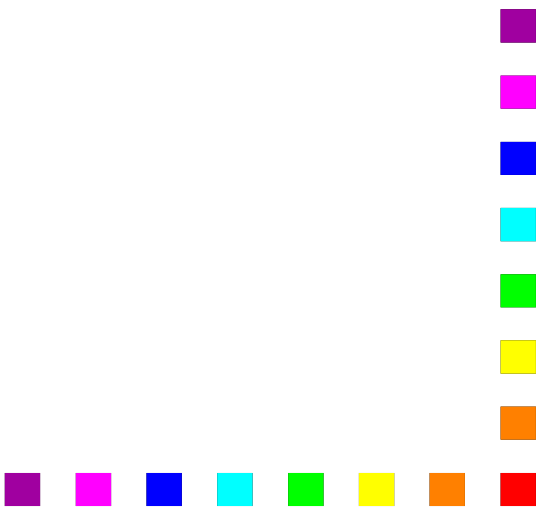
The following symbols are used frequently in picture clauses;

9 (the digit nine) is used to in
at the corresponding posi

X (the character X) is used t

any

assumed decimal point



COBOL 'PICTURE' Clauses

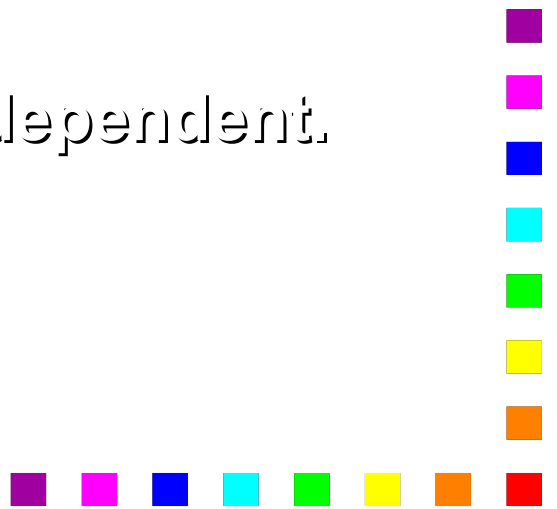
Some examples

```
PICTURE 999      a three digit
PICTURE S999     a three digit
PICTURE XXXX     a four char
PICTURE 99V99    a +ive 'real'
PICTURE S9V9     a +ive/-ive 'r
```

If you wish you can use the abbreviation **PIC**.

Numeric values can have a maximum of 18 (eighteen) digits (i.e. 9's).

The limit on string values is usually system-dependent.



Abbreviating recurring symbols

Recurring symbols can be specified using a 'repeat' factor inside round brackets

PIC 9(6) is equivalent to PIC 999999

PIC 9(6)V99 is equivalent to 999999V99

PICTURE X(10) is equivalent XXXXXXXXXX

PIC S9(4)V9(4) is equivalent S9999V9999

9(18) 999999999999999999

Declaring DATA in COBOL

In COBOL a variable declaration consists of a line containing the following items;

1. A level number.
2. A data-name or identifier.
3. A PICTURE clause.

We can give a starting value to variables by means of an extension to the picture clause called the **value clause**.

```
DATA DIVISION.
WORKING-STORAGE SECTION.
01   Num1              PIC 999    VALUE ZEROS.
01   VatRate           PIC V99    VALUE .18.
01   StudentName       PIC X(10) VALUE SPACES.
```

DATA

Num1	VatRate	StudentName
000	.18	

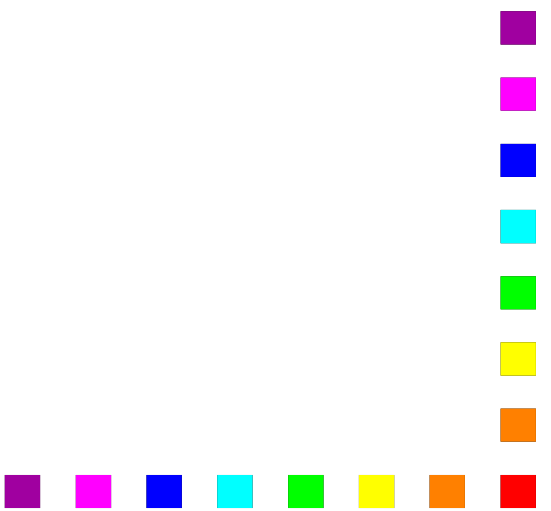
COBOL Literals.

String/Alphanumeric literals are enclosed in quotes and may consists of alphanumeric characters

e.g. "Michael Ryan", "-123" "123.45"

Numeric literals may consist of numerals, the decimal point and the plus or minus sign. Numeric literals are not enclosed in quotes.

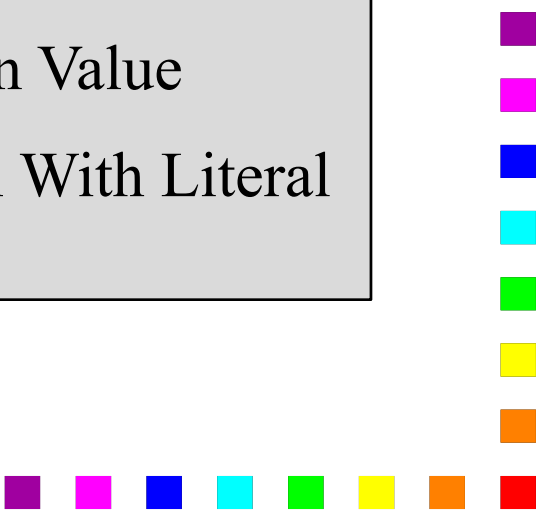
123 123.45 -256 +2987



Figurative Constants

COBOL provides its own, special constants called Figurative Constants.

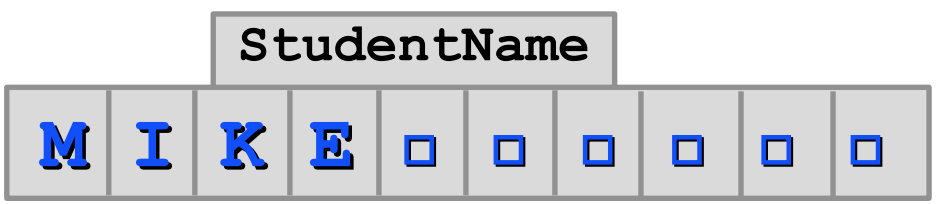
SPACE or SPACES	=	☐
ZERO or ZEROS or ZEROS	=	**0**
QUOTE or QUOTES	=	"
HIGH-VALUE or HIGH-VALUES	=	Max Value
LOW-VALUE or LOW-VALUES	=	Min Value
ALL *literal*	=	Fill With Literal



Figurative Constants - Examples

```
01   GrossPay    PIC 9(5)V99 VALUE 13.5.

MOVE  ZERO     GrossPay.
      ZEROS
      ZEROES
```

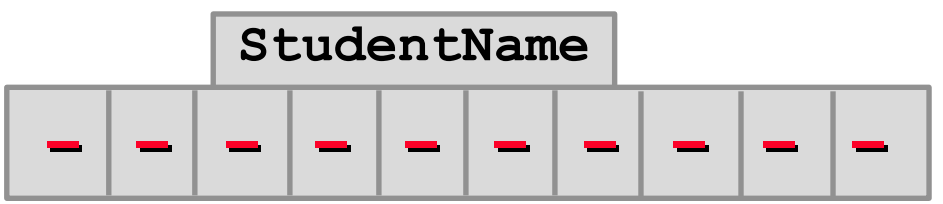

```
01   StudentName    PIC X(10) VALUE "MIKE".

MOVE ALL "-" TO StudentName.
```



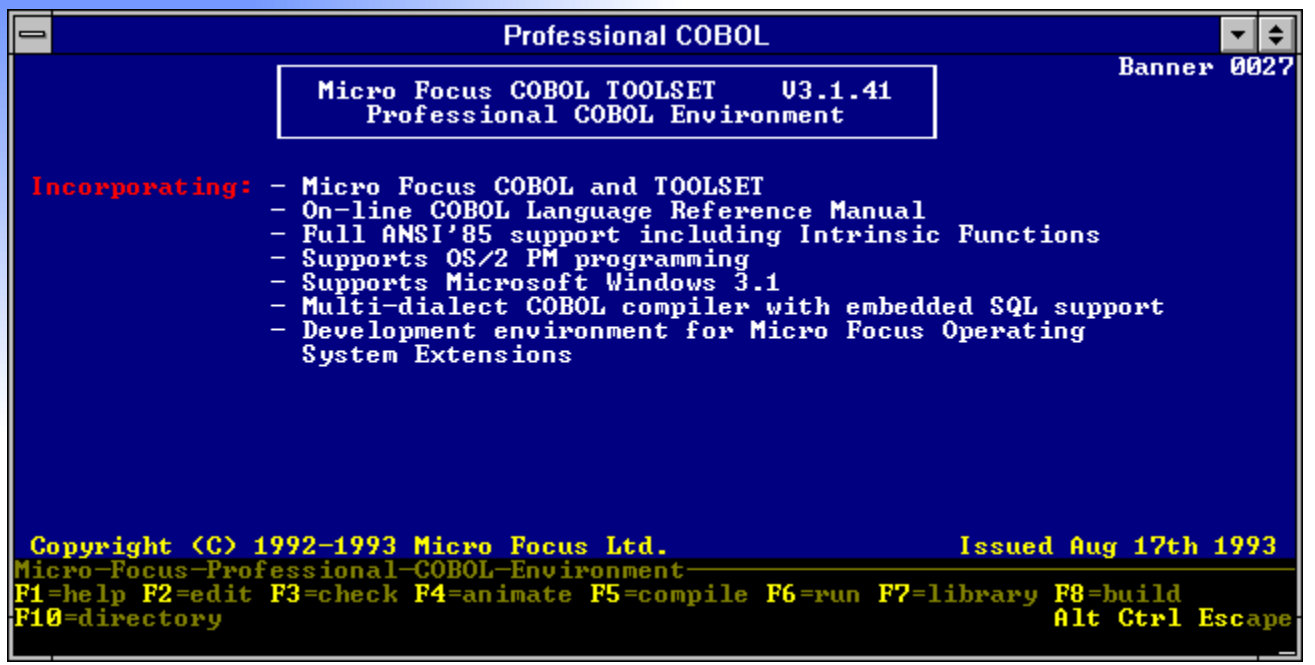
Figurative Constants - Examples

```
01   GrossPay    PIC 9(5)V99 VALUE 13.5.

MOVE  ZERO     GrossPay.
      ZEROS
      ZEROES
```

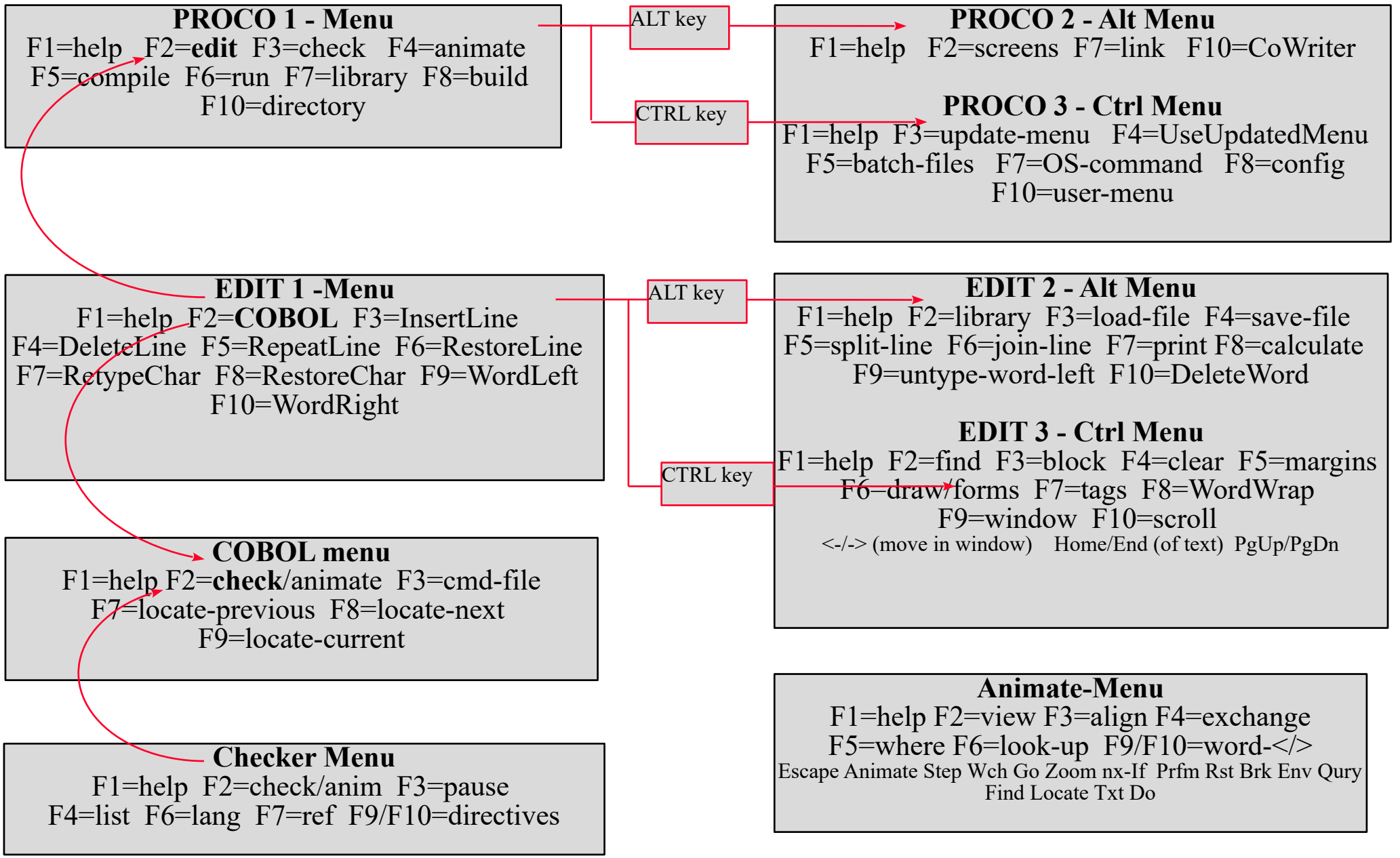

```
01   StudentName    PIC X(10) VALUE "MIKE".

MOVE ALL "-" TO StudentName.
```



Editing, Checking, Compiling, Running



PROCO Menus


PROCO 1 - Menu
F1=help F2=edit F3=check F4=animate
F5=compile F6=run F7=library F8=build
F10=directory
ALT key
CTRL key
PROCO 2 - Alt Menu
F1=help F2=screens F7=link F10=CoWriter
PROCO 3 - Ctrl Menu
F1=help F3=update-menu F4=UseUpdatedMenu
F5=batch-files F7=OS-command F8=config
F10=user-menu
EDIT 1 -Menu
F1=help F2=COBOL F3=InsertLine
F4=DeleteLine F5=RepeatLine F6=RestoreLine
F7=RetypeChar F8=RestoreChar F9=WordLeft
F10=WordRight
ALT key
CTRL key
EDIT 2 - Alt Menu
F1=help F2=library F3=load-file F4=save-file
F5=split-line F6=join-line F7=print F8=calculate
F9=untype-word-left F10=DeleteWord
EDIT 3 - Ctrl Menu
F1=help F2=find F3=block F4=clear F5=margins
F6=draw/forms F7=tags F8=WordWrap
F9=window F10=scroll
<-/-> (move in window) Home/End (of text) PgUp/PgDn
COBOL menu
F1=help F2=check/animate F3=cmd-file
F7=locate-previous F8=locate-next
F9=locate-current
Animate-Menu
F1=help F2=view F3=align F4=exchange
F5=where F6=look-up F9/F10=word-</>
Escape Animate Step Wch Go Zoom nx-If Prfm Rst Brk Env Qury
Find Locate Txt Do
Checker Menu
F1=help F2=check/anim F3=pause
F4=list F6=lang F7=ref F9/F10=directives